UNIVERSITY OF GHANA

COLLEGE OF BASIC AND APPLIED SCIENCES



CAMPUS COMPANION: AN INTEGRATED MOBILE APPLICATION FOR ACADEMIC SCHEDULING, GRADE ANALYTICS AND CAMPUS-LIFE MANAGEMENT FOR GHANAIAN TERTIARY STUDENTS

BY

JUVELYN EKOW SACKEY

(11263517)

A PROJECT WORK SUBMITTED TO THE DEPARTMENT OF COMPUTER SCIENCE, UNIVERSITY OF GHANA, LEGON, IN PARTIAL FULFILMENT OF THE REQUIREMENTS FOR THE AWARD OF BACHELOR OF SCIENCE (BSc) DEGREE IN INFORMATION TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE

OCTOBER, 2026

DECLARATION

I hereby declare that this project, “Campus Companion: An Integrated Mobile Application for Academic Scheduling, Grade Analytics and Campus-Life Management for Ghanaian Tertiary Students”, is the result of my own work, carried out under the supervision of Mr. Solomon Mensah in the Department of Computer Science, University of Ghana.

Except where other people's work has been duly acknowledged by reference, this work is my own, and it has not been submitted, in whole or in part, for the award of any other degree, diploma or qualification in this or any other institution.

STUDENT

Name: Juvelyn Ekow Sackey

Signature: .....

Date: .....

SUPERVISOR

Name: Mr. Solomon Mensah

Signature: .....

Date: .....

ABSTRACT

Context

University students in Ghana keep track of timetables, grades, announcements, events and revision materials using several unconnected tools, such as paper diaries, general calendar apps and social media groups. These tools are not designed around their institutions' academic rules, such as the University of Ghana grading scale, under which a mark of 45–49 does not count as a pass.

Aim

This project set out to design, build and evaluate Campus Companion, a mobile application that brings academic scheduling, grade analytics and campus-life services together in one secure application for Ghanaian university students.

Method

Requirements came from the developer's own analysis of students' academic and campus-life needs; no interviews or surveys were conducted. The system consists of a Flutter mobile application, a Node.js and Express REST API and a PostgreSQL database, built iteratively one feature at a time. It was tested at three levels (offline unit tests, database integration tests and end-to-end API smoke tests), and mutation testing was used to check that the tests detect deliberately introduced faults.

Result

In the final test runs on 5 and 6 November 2026, all 449 Flutter tests, 119 backend unit tests, 14 database tests and 212 API smoke checks passed, and static analysis reported no issues. Running the complete application exposed three interface defects that the tests had missed; these were fixed and are now covered by new tests. Most objectives were met at the working-prototype level. Google Sign-In, push-notification delivery, the street map and the Gemini-backed assistant were confirmed on a physical Android phone against the local backend; the assistant's answers have not been formally evaluated, and no usability evaluation with students has been carried out.

Conclusion

The project shows that a single, institution-aware application can bring students' academic and campus-life tasks together, with its behaviour supported by repeatable automated tests. The

backend has been deployed on free hosting plans; wider device verification and evaluation with users remain as future work.

Keywords

mobile application; academic scheduling; grade point average; AI assistant; Ghana; Flutter

DEDICATION

I dedicate this project to God Almighty for His grace, strength and guidance throughout my academic journey. I also dedicate this work to my parents and family for their unconditional love, sacrifices, prayers, encouragement and unwavering support throughout my education.

ACKNOWLEDGEMENT

First and foremost, I express my sincere gratitude to God Almighty for His grace, strength, wisdom and guidance throughout my academic journey and during the development and completion of this project.

I extend my heartfelt appreciation to my supervisor, **Mr. Solomon Mensah**, for his guidance, constructive criticism, patience and continuous support throughout this project. His knowledge, advice and encouragement provided valuable direction during the planning, design, development and documentation of Campus Companion.

My sincere thanks also go to all lecturers in the Department of Computer Science, University of Ghana, for imparting the knowledge, skills and professional values that have contributed to my academic and personal development.

I am deeply grateful to my parents and family for their unconditional love, prayers, sacrifices, encouragement and financial support throughout my education. Their belief in me has remained a constant source of motivation and strength.

I also wish to acknowledge my colleagues, friends and classmates for their encouragement, helpful suggestions and support throughout the development of this project.

Finally, I sincerely appreciate everyone who contributed in diverse ways, directly or indirectly, towards the successful completion of this project. May God richly bless you all.

TABLE OF CONTENTS

DECLARATION i
ABSTRACT ii
DEDICATION iv
ACKNOWLEDGEMENT v
TABLE OF CONTENTS vi
LIST OF FIGURES ix
LIST OF TABLES x
LIST OF ABBREVIATIONS xi

CHAPTER ONE: INTRODUCTION 1
1.1 Background and Motivation 1
1.2 Statement of Problem 1
1.3 Scope of the Study 2
1.4 Research Objectives 3
1.4.1 Global Objective 3
1.4.2 Specific Objectives 3
1.5 Research Contribution 4
1.6 Organization of the Study 4

CHAPTER TWO: LITERATURE REVIEW 5
2.1 Introduction 5
2.2 Mobile Applications for Academic Scheduling and Student Life Management 5
2.3 Mobile Learning and Smartphone Use Among Ghanaian University Students 6
2.4 Authentication and Security in Mobile Applications 6
2.5 Push Notification Architectures in Mobile Systems 7
2.6 AI-Assisted Virtual Assistants and Chatbots in Education 8
2.7 Review of Existing Systems 8
2.8 Summary and Research Gap 9

CHAPTER THREE: RESEARCH METHODOLOGY 11
3.1 Requirements Gathering Approach 11
3.2 Requirement Analysis 11
3.2.1 Functional Requirements 11
3.2.2 Non-Functional Requirements 12

3.3 System Design 13
3.3.1 System Architecture 13
3.3.2 Use Case Design 14
3.3.3 Database Design 15
3.3.4 Sign-in Design 17
3.4 Feature Selection 17
3.5 Experimental Setup 18
3.5.1 Development Tools and Environment 18
3.5.2 Development Methodology 18
3.5.3 Testing Strategy 19
3.5.4 Evaluation Criteria 19
3.5.5 Statistical Analysis 20

CHAPTER FOUR: EXPERIMENTAL RESULT AND DISCUSSION 21
4.1 Introduction 21
4.2 Overview of the Implemented System 21
4.3 Results of Feature Implementation 22
4.3.1 Authentication and Onboarding 22
4.3.2 Academic Management 23
4.3.3 Quiz and Past-Paper Features 25
4.3.4 AI Assistant and Tutor 26
4.3.5 Campus-Life Features 27
4.4 Automated Testing Results 29
4.4.1 Summary of Test Results 29
4.4.2 Flutter Automated Tests 29
4.4.3 Backend Unit Tests 30
4.4.4 Database Integration Tests 31
4.4.5 API Smoke Tests 31
4.4.6 Static Analysis 32
4.4.7 Defects Found by Running the Application 32
4.5 Evaluation Against Project Objectives 32
4.6 Discussion of Findings 34
4.6.1 Integration of Academic and Campus Services 34
4.6.2 Consistency of Academic Calculations 34
4.6.3 AI Assistance and Fallback Behaviour 34
4.6.4 Reliability of Notification and Sign-in Integrations 35
4.6.5 Local-First Operation and Synchronisation 36

4.6.6 Technical Quality and Test Coverage 36
4.7 Outstanding Issues and Deployment Readiness 37
4.8 Limitations of the Evaluation 38
4.9 Chapter Summary 39

CHAPTER FIVE: CONCLUDING REMARKS 40
5.1 Summary of the Study 40
5.2 Contribution of the Study 40
5.3 Limitations 41
5.4 Recommendations for Future Work 42
5.5 Conclusion 42

REFERENCES 44

LIST OF FIGURES

Figure 3.1: System architecture of Campus Companion 14
Figure 3.2: Use case diagram 15
Figure 3.3: Entity-relationship diagram of the Campus Companion database 16
Figure 3.4: Google Sign-In sequence 17
Figure 4.1: Onboarding, sign-in and guest mode 23
Figure 4.2: Academic features 25
Figure 4.3: Campus-life features 28

LIST OF TABLES

Table 2.1: Summary of existing systems and related works 8
Table 3.1: Functional requirements 11
Table 3.2: Non-functional requirements 12
Table 3.3: System architecture layers 13
Table 3.4: Database tables in the Campus Companion schema 15
Table 3.5: Development tools and environment 18
Table 4.1: Summary of automated testing results 29
Table 4.2: Assessment of the specific objectives 32
Table 4.3: Outstanding issues and required verification 37

LIST OF ABBREVIATIONS

API	Application Programming Interface
APK	Android Package (Kit)
CRUD	Create, Read, Update, Delete
FCM	Firebase Cloud Messaging
GPA	Grade Point Average
ICS	iCalendar file format (.ics)
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large Language Model
REST	Representational State Transfer
SQL	Structured Query Language
UG	University of Ghana
UI/UX	User Interface / User Experience

CHAPTER ONE
INTRODUCTION

1.1 Background and Motivation

University students in Ghana handle many routine academic and campus tasks: following lecture and examination timetables, keeping track of their grades and grade point average (GPA), keeping up with announcements, finding buildings on campus, registering for events and clubs, and revising for quizzes and examinations. In practice these tasks are spread across several unconnected tools, including paper diaries, general-purpose calendar apps, noticeboards, WhatsApp groups and spreadsheets. None of these tools is designed around the way a Ghanaian university works, and none of them knows institution-specific rules such as the University of Ghana grading scale.

Mobile devices are already part of how Ghanaian students study, although much of this use happens outside official university systems (Adzifome & Agyei, 2022; Afful & Boateng, 2023). At the same time, cross-platform frameworks such as Flutter make it practical for one developer to build a single application that runs on Android and the web from one codebase. These two observations motivated this project: to bring the scattered tools that students already rely on into one purpose-built mobile application.

Local rules matter. Under the University of Ghana scale, a mark of 45–49 is graded E and does not count as a pass, so a general-purpose planner with its own grading assumptions can give a student the wrong picture of their standing. An application is only useful to these students if it follows the rules of their own institution.

1.2 Statement of Problem

Ghanaian university students have no single application that brings their routine academic and campus-life tasks together while following their institution's rules, and the literature reviewed in Chapter Two found no evaluated system that does so. This fragmentation causes three problems. First, information is duplicated or missed: a timetable kept on paper does not send reminders, and a notice on a board may never reach a student who does not pass it. Second, tracking grades by hand or with generic tools is error-prone; a student may, for example, count a mark of 47 as a pass when it is not. Third, there is no single place to get a quick answer to

everyday questions such as “when is my next class?”, “what is happening this week?” or “where is this building?”.

This project addresses the problem by building Campus Companion, a mobile application with a supporting backend that brings timetables, examinations, GPA analytics on the University of Ghana scale, announcements, events, clubs, campus locations, quizzes, a digital student ID and an AI assistant together behind one account.

1.3 Scope of the Study

The study covers the design, implementation and testing of the two parts of Campus Companion: a Flutter mobile application, and a Node.js and Express REST API backed by a PostgreSQL database. The following are within scope:

- Registration, login (including Google Sign-In) and role-based access for students, lecturers and administrators, plus a guest mode for browsing campus information without an account.
- A timetable and examination module that stores its data on the device, with reminders and calendar (.ics) import and export.
- GPA and CWA analytics using the University of Ghana grading scale, including the honours class and a calculator for the average needed to reach a target class.
- Campus announcements, events (creation, registration, cancellation and deletion, with ownership rules) and a clubs directory.
- A campus-location directory with a map view.
- Quizzes written by lecturers, including questions extracted from past-paper PDFs with the help of a large language model (LLM).
- An AI assistant and an AI tutor that use Google Gemini when it is configured and a local rule-based engine otherwise, keep answering when some backend data cannot be reached, and, in the tutor's case, can answer from lecture slides the student uploads as a PDF.
- A digital student ID card with a QR code.
- An administrator panel for creating lecturer and administrator accounts.

- Push notifications for announcements through Firebase Cloud Messaging, delivered to a physical phone and enabled on the deployed server.

The following are outside the scope of this study: payment of dues, shuttle timetables and live library information, which would need data and payment arrangements from the University (the application does not show invented versions of them); Apple sign-in and in-app password reset; an iOS version (the application targets Android and the web); and production deployment, including the final configuration that push notifications and Google Sign-In need before they can be used. These limitations are discussed again in Chapter Five.

1.4 Research Objectives

1.4.1 Global Objective

The global objective of this study is to design, implement and evaluate a mobile application that brings academic scheduling, grade analytics and campus-life services together in one secure platform for Ghanaian university students, following the academic rules of their institutions.

1.4.2 Specific Objectives

1. To design and implement a cross-platform mobile application (Flutter) and a REST backend (Node.js, Express and PostgreSQL) with role-based authentication for students, lecturers and administrators, including sign-in with Google.
2. To implement a timetable, examinations and alerts module that works without a constant server connection and supports calendar (.ics) import and export and reminders.
3. To implement GPA and CWA analytics that apply the University of Ghana grading scale, with the mobile application and the backend using one shared definition of the scale.
4. To implement campus-life features (events, clubs, campus locations, quizzes and a digital student ID) within the same application.
5. To implement an AI assistant that answers academic and campus questions using a cloud language model where available and a local rule engine otherwise, and that keeps working when some backend data sources are unavailable.

6. To integrate push notifications for campus announcements through Firebase Cloud Messaging.
7. To test the system through automated unit, database and end-to-end tests, and to check the tests themselves through mutation testing.

1.5 Research Contribution

The main contribution of this project is a working application that combines a student's own academic information with campus-wide services and applies the University of Ghana's grading rules, a combination that the general-purpose planners and the systems reviewed in Chapter Two do not offer (Table 2.1). The project also shows a practical way to make an AI assistant degrade gracefully: when one of its data sources cannot be reached, it answers from the others and tells the student what it could not access. Finally, it documents a testing approach suited to a student software project: three levels of automated tests, a single shared definition of the grading scale that both the mobile application and the backend are tested against, and mutation testing to confirm that the tests detect real faults.

1.6 Organization of the Study

Chapter Two reviews literature on mobile applications for student life, mobile learning in Ghana, authentication and push-notification design, and AI assistants in education, and compares existing systems. Chapter Three describes the methodology: how the requirements were gathered and analysed, the design of the system, the features selected, and the development and testing approach. Chapter Four presents the implemented system and the test results and evaluates them against the objectives in Section 1.4. Chapter Five summarises the study, states its limitations and recommends future work.

CHAPTER TWO
LITERATURE REVIEW

2.1 Introduction

This chapter reviews literature relevant to the design of Campus Companion. It is organised around the system's main technical and functional concerns: mobile applications built to support students' academic and campus life (Section 2.2); mobile and smartphone use among Ghanaian university students, which motivates and contextualises this study (Section 2.3); authentication and security practices for mobile applications and the standards behind them, relevant to the system's sign-in design (Section 2.4); push-notification architectures, relevant to announcement delivery (Section 2.5); and AI assistants and chatbots in education, relevant to the system's AI assistant and tutor (Section 2.6). Section 2.7 compares existing systems with the one built in this study, and Section 2.8 summarises the chapter and identifies the gap this study addresses.

2.2 Mobile Applications for Academic Scheduling and Student Life Management

Several studies try to bring separate parts of a student's life, such as tasks, deadlines, wellbeing and campus services, into a single mobile tool. Zhao and Pardo (2017) describe the user-centred design of myUniMate, an application that helps first-year students track and reflect on their university life. It offers a to-do list with reminders, a calendar view of those tasks, mood tracking, user-defined measures and simple charts. In a four-week trial with 11 first-year engineering students, the students' perceived competence and relatedness support and their learning motivation increased, while the autonomy support they attributed to the application decreased. myUniMate is a personal organiser: it holds no class timetable or grades and offers no campus-wide services.

Baras et al. (2018) describe an application that keeps a student's schedules and deadlines in one place and adds virtual-tutor features, with a particular focus on supporting students' mental health alongside their academic success. Like myUniMate, it focuses on the individual student's organisation, not on campus-wide information.

Wang and Yan (2018) take the opposite approach. They describe an Android “mobile smart campus” system that delivers a campus's internet services through one application. This is closer to the campus-life side of Campus Companion (events, clubs, locations and

announcements) than to its academic side. In the papers reviewed here, personal academic organisers and campus-service systems are rarely combined in one evaluated system. Bringing them together behind a single account is one of the gaps this project addresses (Section 2.8).

The systems reviewed were usually evaluated with small groups at a single institution over short periods (myUniMate's trial involved 11 students for four weeks), and none reports a long-running production deployment. This is typical of student-facing academic tools, which are often built as single-institution or final-year projects, as this one is.

2.3 Mobile Learning and Smartphone Use Among Ghanaian University Students

Ghana-specific studies show that mobile devices are already part of Ghanaian university students' academic lives, but largely outside formal academic systems. Afful and Boateng (2023) studied distance-education students at the University of Ghana and the University of Education, Winneba, using a decomposed Theory of Planned Behaviour and an explanatory sequential mixed-methods design to examine how students' attitudinal, social and control beliefs affect their mobile learning. They found that students mostly used their phones for calls and text messages, and recommended training in retrieving information on mobile devices. Adzifome and Agyei (2022), studying students at a Ghanaian university, reported that the facilities needed for mobile learning were largely available and students' experiences of it were positive, but that much of this mobile learning took place outside the institution's learning management system.

These findings relate directly to the problem in Section 1.2. Ghanaian students have mobile devices and use them for study, but the tools they use are general-purpose and are not built around their institution's academic structure, such as its grading scale, calendar, courses and clubs. Neither study evaluates an application designed for a particular institution; instead, they describe the gap such an application would need to fill. Campus Companion responds to this gap, most clearly by applying the University of Ghana grading scale instead of a generic one (Sections 1.2 and 3.3.3).

2.4 Authentication and Security in Mobile Applications

Campus Companion supports three roles (student, lecturer and administrator) and two ways of signing in (password and Google), so its authentication design draws on research into OAuth on mobile devices. Chen et al. (2014) examined how 149 mobile applications used OAuth and found that 89 of them (59.7%) were implemented incorrectly and were therefore vulnerable,

often because they trusted information on the client as if the server had verified it. Their main lesson is that the application's own backend, not the mobile client, must verify an identity token before trusting it. Campus Companion follows this pattern: the Google ID token obtained on the phone is exchanged for a Firebase ID token, which the backend verifies itself through the Firebase Admin SDK before it issues its own session token (Figure 3.4).

Several standards underpin this design. Google Sign-In builds on OAuth 2.0, the authorisation framework whose misuse Chen et al. (2014) document (Hardt, 2012). After sign-in, the backend issues its own session token as a JSON Web Token: a compact, digitally signed set of claims that the server can check on every request without keeping session state (Jones et al., 2015). That suits the REST architectural style described by Fielding (2000), in which each request carries everything the server needs to process it. Passwords are never stored; the backend keeps only a bcrypt hash, a salted and deliberately slow hash whose cost can be raised as computers become faster (Provos & Mazières, 1999). These choices respond to the OWASP Top 10, which ranks broken access control as the most serious web application security risk and lists identification and authentication failures among the others (OWASP Foundation, 2021).

The same principle runs through the rest of the design: the backend trusts role and identity information only after verifying it on the server, never because the client says so. For example, a user who registers themselves is always created as a student, whatever role value the client sends. This guards against exactly the kind of client-trust weakness that Chen et al. (2014) describe.

2.5 Push Notification Architectures in Mobile Systems

Campus Companion delivers announcements through Firebase Cloud Messaging (FCM), the push service most commonly used on Android. The most relevant recent study of FCM in practice is by Samarin et al. (2024). They examined 21 popular Android secure-messaging applications that use FCM and found that 11 leaked user-identifying metadata through push notifications and 4 leaked actual message content; none of the affected applications disclosed this in its privacy policy.

This finding shaped a design decision in Campus Companion: the FCM payload used for announcement notifications carries no announcement content, only a signal that new content is available, which the client then fetches from the authenticated announcements endpoint.

Keeping content out of the payload avoids the class of leak Samarin et al. (2024) document, at the cost of one extra request when the notification is opened.

2.6 AI-Assisted Virtual Assistants and Chatbots in Education

Two systematic reviews give a broad picture of chatbot use in education. Okonkwo and Ade-Ibijola (2021), reviewing 53 articles, identify educational chatbots used for teaching, administration, assessment, advisory work and research; the benefits reported include immediate help for students, and the challenges include ethical, programming and maintenance issues. Wollny et al. (2021), reviewing 74 publications drawn from an initial pool of 2,678, frame the field's open research challenges around chatbot evaluation, mentoring and adaptation to the learner.

A study set directly in the Ghanaian context, and therefore the most relevant single source for this project's AI assistant, is Essel et al. (2022). The authors built KNUSTbot, a chatbot deployed in a multimedia-programming course at Kwame Nkrumah University of Science and Technology, motivated by the high instructor-to-student ratio common in Ghanaian higher education: a chatbot can give large cohorts immediate answers to routine questions that would otherwise wait for an instructor. In a pretest–posttest study with 68 undergraduates allocated across a 2×2 experimental design, students who interacted with the chatbot performed better academically than those who interacted with the instructor.

The result of Essel et al. (2022) supports including an AI assistant in Campus Companion, but the sample (68 students in one course at one institution) is small, so it is best read as one promising result, not as proof that any chatbot improves learning. The studies reviewed here also focus on whether chatbots help learning, not on how a chatbot should behave when a service it depends on is unavailable. Campus Companion's assistant draws on several backend data sources (grades, events, alerts and campus locations) and on data stored on the device (today's classes and exams). It was designed to tell the student when one of these sources cannot be reached, and to keep answering from the others.

2.7 Review of Existing Systems

Table 2.1 compares Campus Companion with existing systems a Ghanaian student might use and with the related systems reviewed in this chapter.

Table 2.1: Summary of existing systems and related works

System / work	Description	Strengths	Limitations for this context
University of Ghana Sakai LMS (University of Ghana, n.d.)	The University's learning management system for course materials, assignments and notices.	Official; used across the University's courses.	Centred on course delivery; no GPA analytics on the UG scale, events, clubs or campus map.
MyStudyLife (MyStudyLife, n.d.)	Commercial student planner for timetables (weekly or rotating), exams, tasks and reminders, with grade tracking; syncs across devices and works offline.	Mature, multi-platform, usable offline.	Not tied to any institution, so it has no institution-specific grading rules or campus services; some features need a paid subscription.
myUniMate (Zhao & Pardo, 2017)	Research prototype for first-year students: to-do list with reminders, mood tracking and user-defined measures.	Designed and evaluated with students.	Personal organiser only; no timetable, grades or campus services; 11-student trial.
Mobile smart campus (Wang & Yan, 2018)	Android system delivering a campus's internet services through one application.	Brings campus services into one app.	Campus services only; no personal academic data.
KNUSTbot (Essel et al., 2022)	Chatbot teaching assistant for one course at a Ghanaian university.	Ghanaian context; students using it performed better.	A single-course chatbot, not an integrated campus application.
Campus Companion (this study)	Mobile application combining timetable, exams, UG-scale GPA analytics, announcements, events, clubs, campus map, quizzes, an AI assistant and a digital ID.	Personal academic data and campus services behind one account; institution-aware grading.	Push notifications and Google Sign-In await configuration; not yet evaluated with users (Section 5.3).

No single system in Table 2.1 combines a student's own academic data, institution-specific grading, campus-wide services and an AI assistant in one application, which is the combination this study builds.

2.8 Summary and Research Gap

The literature reviewed in this chapter points to three conclusions. First, applications that organise a student's personal academic life have been studied (Zhao & Pardo, 2017; Baras et al., 2018), as have applications that bring campus services together (Wang & Yan, 2018), but the two are rarely combined in one system behind a single account (Table 2.1). Second, Ghanaian students already use mobile devices for study, mostly outside formal academic systems (Afful & Boateng, 2023; Adzifome & Agyei, 2022), yet no application built around their institution's rules, such as its grading scale, is evaluated in this literature. This is the gap identified in Section 1.2. Third, mobile authentication security (Chen et al., 2014) and push-notification privacy (Samarin et al., 2024) are well studied, and AI chatbots show early promise

in a Ghanaian setting (Essel et al., 2022), but none of the systems reviewed brings these concerns together in one academic and campus-life application. Chapter Three describes how Campus Companion was designed and built to fill this gap.

CHAPTER THREE
RESEARCH METHODOLOGY

This chapter describes how Campus Companion was specified, designed, built and evaluated. Because this is a software development project rather than a data-modelling study, the template's methodology headings (dataset description, preprocessing, oversampling, feature selection, model training and statistical tests) have been adapted to the corresponding stages of building and evaluating a software system: gathering and analysing requirements, designing the system, selecting its features, and setting up its development and testing.

3.1 Requirements Gathering Approach

The requirements for Campus Companion were identified primarily through personal analysis of the academic and campus-life needs of university students. The requirements-gathering process involved examining common challenges associated with accessing academic information, organising study activities, locating campus resources and keeping up with university announcements and events.

Based on this analysis, the functional and non-functional requirements of the application were established. These requirements informed the design and development of the system's major modules, including timetable management, examination support, grade tracking, quizzes, campus information, events, clubs and the AI-powered assistant.

No formal interviews or questionnaire-based surveys were conducted as part of the requirements-gathering process. Consequently, the requirements presented in this study reflect the developer's analysis and design decisions rather than findings from a formal participant study.

3.2 Requirement Analysis

The gathered requirements were organised into functional requirements, which state what the system must do, and non-functional requirements, which state the qualities it must have (Sommerville, 2016). Tables 3.1 and 3.2 list them; Chapter Four checks the implemented system against them.

3.2.1 Functional Requirements

Table 3.1: Functional requirements

ID	Requirement	Users
FR1	The system shall let a student register with a name, email address and password; self-registration always creates a student account.	Student
FR2	The system shall authenticate users by email and password, and by Google Sign-In.	All users
FR3	The system shall let an administrator create lecturer and administrator accounts.	Administrator
FR4	The system shall let a user choose their institution, which sets the grading scale and branding the app uses.	All users
FR5	The system shall let a student add, edit, delete, import (.ics or CSV) and export (.ics) timetable classes, stored on the device, with class reminders.	Student
FR6	The system shall let a student record exams, see countdowns and receive exam reminders.	Student
FR7	The system shall compute GPA or CWA from the student's grades on the institution's scale, with the honours class and a calculator for the average needed to reach a target class.	Student
FR8	The system shall show campus announcements and let an administrator post them, with push notifications once push delivery is enabled.	All users, Administrator
FR9	The system shall let users browse and RSVP to events; lecturers and administrators create events; a lecturer deletes only their own events, an administrator any event.	All users, Lecturer, Administrator
FR10	The system shall let users search clubs and join or leave them.	Student
FR11	The system shall list campus locations by category, show them on a map, and open them in a maps app.	All users
FR12	The system shall let lecturers and administrators create quizzes, including from a past-paper PDF, and let students take them and see their scores.	Lecturer, Administrator, Student
FR13	The system shall answer academic and campus questions through an AI assistant and tutor, and keep answering when some backend data sources are unreachable.	Student
FR14	The system shall display a digital student ID card with a QR code.	Student
FR15	The system shall let a visitor browse announcements, events, clubs and campus locations without an account, and shall ask them to sign in before any action that needs one.	Guest

3.2.2 Non-Functional Requirements

Table 3.2: Non-functional requirements

Requirement	Description
Security	Passwords are hashed with bcrypt before storage; only campus-wide information (announcements, events, clubs and locations) can be read without a valid JSON Web Token, and every other request needs one; roles are enforced on the server, not only hidden in the interface; request bodies are validated before use.
Privacy	Push notifications carry no announcement content (Section 2.5); the optional profile details for the ID card stay on the device.
Resilience	The timetable and exams work without the server; the AI assistant reports which data sources it could not reach instead of failing outright.
Correctness	The grading scale is defined once and shared by the server and the app, so the two cannot disagree on a grade.

Requirement	Description
Usability	Core features are reachable from the dashboard through labelled shortcuts, and forms show their errors in place.
Portability	One Flutter codebase runs on Android and the web, against one REST API.
Maintainability	The code is layered (screens, state, repositories; routes, controllers, services) and covered by automated tests at three levels (Section 3.5.3).

3.3 System Design

3.3.1 System Architecture

Campus Companion follows a layered client–server architecture, summarised in Table 3.3 and shown in Figure 3.1.

Table 3.3: System architecture layers

Layer	Technology	Responsibility
Mobile client	Flutter, Riverpod (state management), Dio (HTTP)	Renders the UI; holds local-first data (timetable, exams) on-device; calls the REST API for server-backed features.
Backend API	Node.js, Express, Zod (validation)	Stateless REST API; authentication/authorisation, business rules (grading scale, event-deletion ownership), request validation.
Database	PostgreSQL	Durable relational storage for the entities in Table 3.4.
Identity provider	Firebase Authentication	Issues and verifies Google Sign-In identity tokens, shared with the push-notification Firebase app.
Push delivery	Firebase Cloud Messaging	Delivers announcement notifications to registered device tokens; payload carries no announcement content (Section 2.5).
AI assistant backend	Google Gemini API (optional), local rule engine (fallback)	Answers assistant/tutor queries; falls back to an offline rule engine when no API key is configured or the call fails.

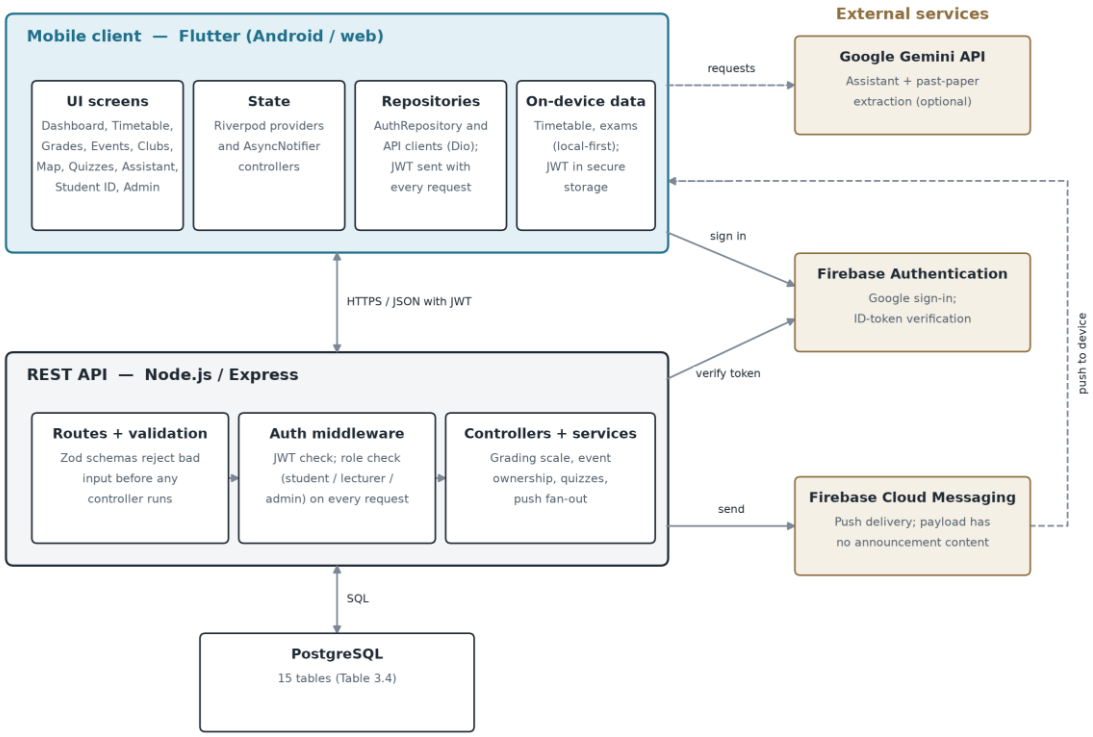


Figure 3.1: System architecture of Campus Companion

The layers communicate only through defined interfaces. The mobile client never queries the database directly, and the same REST API serves the Flutter application whether it runs on Android or as a web build. Role-based access control is enforced by the backend on every request that needs it, not just hidden in the mobile interface, so a client cannot gain extra privileges by bypassing a screen (Section 2.4).

3.3.2 Use Case Design

Figure 3.2 shows the system's three actors and what each can do. Students register themselves and use the academic and campus-life features. Lecturer and administrator accounts cannot be self-registered: an administrator creates them (FR3). Lecturers create events and quizzes and may delete only the events they created; administrators can also delete any event or quiz, post announcements, record grades and create staff accounts. These rules are enforced by the backend's role checks, and the diagram was drawn from those checks rather than from the interface.

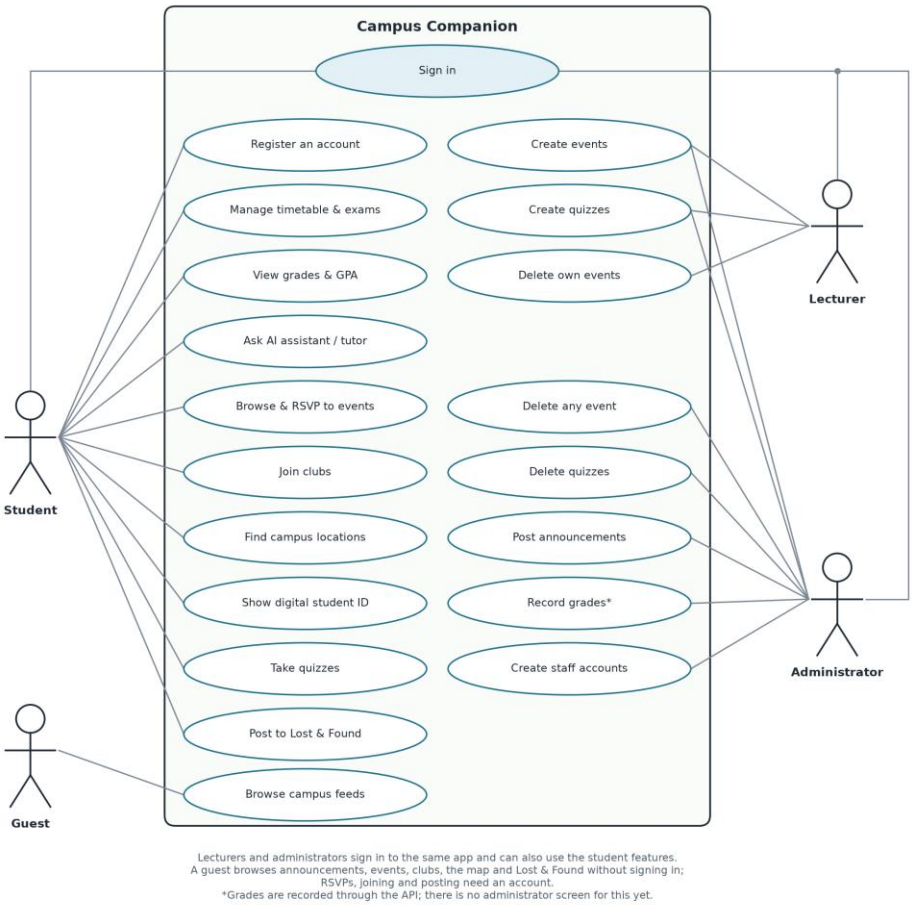


Figure 3.2: Use case diagram

3.3.3 Database Design

The system's data is held in a PostgreSQL database of sixteen tables (`backend/db/schema.sql`), listed in Table 3.4; Figure 3.3 shows how they relate.

Table 3.4: Database tables in the Campus Companion schema

Table	Purpose
users	One row per account (student, lecturer or admin), holding the password hash and/or a linked Google UID, role, and push preferences.
courses	Canonical course records (code, title, lecturer, credit hours), referenced by schedules, grades and quizzes to avoid free-text duplication.
schedules	A user's timetabled class slots (day, start/end time, room), used server-side only as an optional cloud-backup layer; the app itself is local-first (Section 1.3).
grades	A score per user per course per semester; the letter grade is a database-generated column derived from the score using the University of Ghana scale, so it cannot drift from the score it is derived from. A flag marks results a student entered themselves, which never overwrite an official grade.
clubs / club_memberships	Club records and the many-to-many membership junction between users and clubs.
events / event_registrations	Campus events (optionally linked to a club and to the user who created them) and the many-to-many registration junction between users and events.

Table	Purpose
announcements	Broadcast messages posted by staff, categorised (general, academic, emergency) and timestamped for sync.
quizzes / quiz_questions / quiz_attempts / quiz_attempt_answers	A course-anchored quiz, its questions (multiple-choice or open-ended), and each student's attempt and per-question answers.
device_tokens	One row per installed app instance registered for push notifications, keyed by a unique FCM token.
locations	Named campus points of interest with a category and coordinates, used by the campus-map feature.
lost_found_items	Posts on the shared Lost & Found board: whether the item was lost or found, a title, optional place, details and contact note, an open or returned status, the posting user, and an optional photo, stored in the database because the free host's disk is wiped whenever the service restarts.

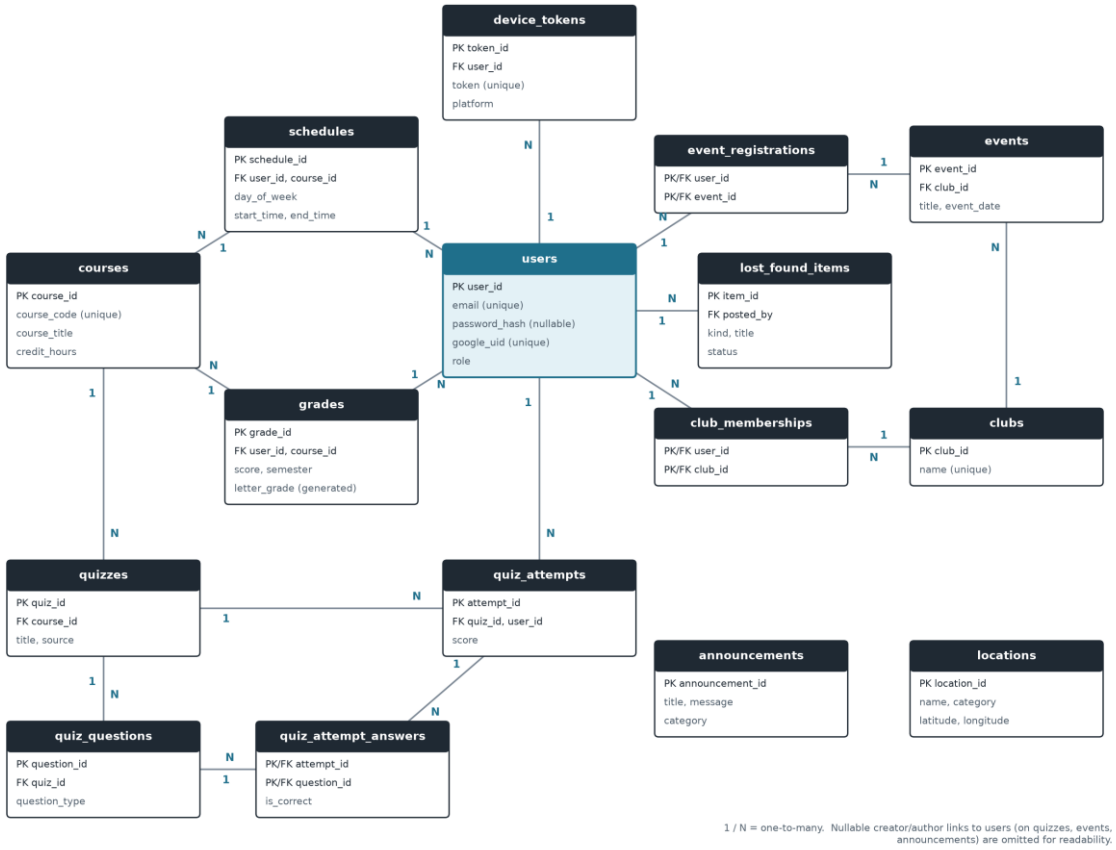


Figure 3.3: Entity-relationship diagram of the Campus Companion database

Data is checked before it reaches the database and again inside it. Every incoming API request body is validated against a schema written with the `zod` library before any controller code runs; for example, `registerSchema` trims and lower-cases the email address and enforces a minimum password length before a user row is created (`backend/src/controllers/authController.js`). A unique index on `LOWER(email)` makes email matching case-insensitive. The letter grade is generated from the numeric score

inside the database (a `GENERATED ALWAYS AS` column), so it can never disagree with the score. Calendar files that a student imports (`.ics` or CSV) are converted into the application's own schedule format by a tested parser before any other feature uses them.

3.3.4 Sign-in Design

Figure 3.4 traces one request path through the layers in detail: Google Sign-In. The mobile app never tells the backend who the user is. It forwards a Firebase ID token, the backend verifies that token with Firebase itself, and only then looks up, links, or creates the account and issues the app's own session token (JWT). This is the server-side verification pattern recommended by Chen et al. (2014), discussed in Section 2.4.

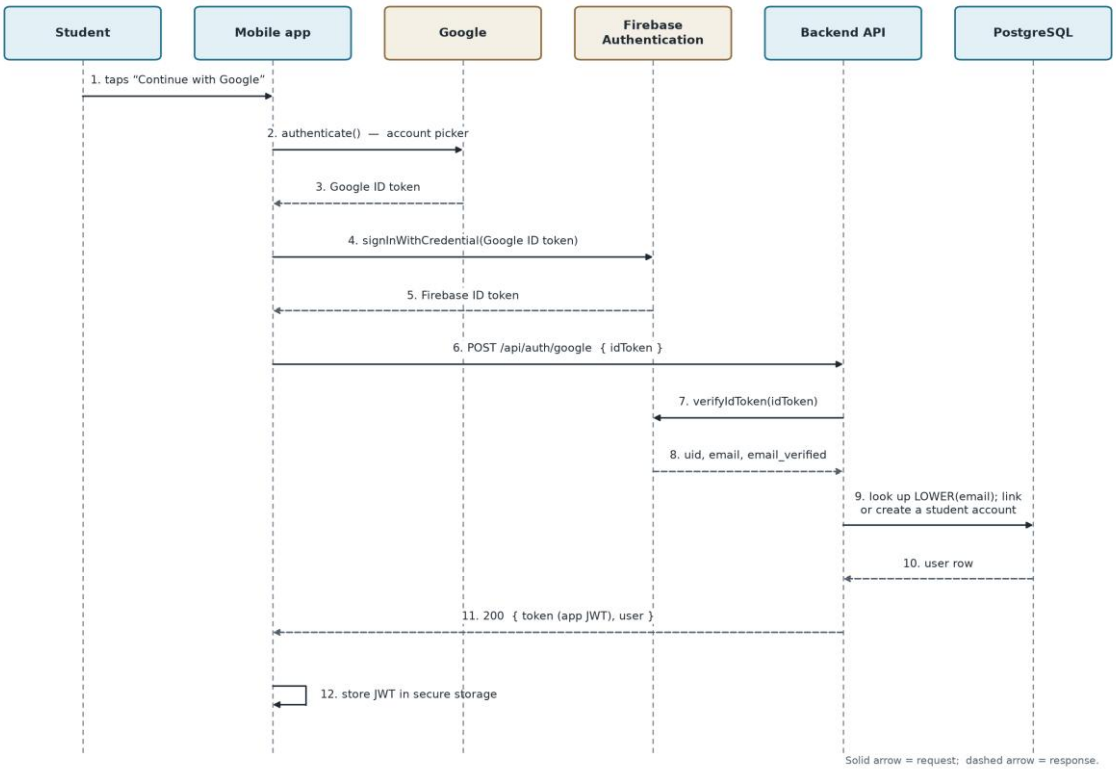


Figure 3.4: Google Sign-In sequence

3.4 Feature Selection

The features were chosen from the requirements in Section 3.2, taking into account the time available, external dependencies and their relevance to the problem in Section 1.2; Section 1.3 lists the resulting scope. The scope was revised as the project progressed. For example, the administrator panel was built before push notifications because it needed no external service. Push notifications and Google Sign-In were built and tested but left switched off, because they depend on Firebase console configuration outside the application's code. Screens for dues,

shuttles, library seats, past papers and an academic calendar were built at first on sample data, but were removed before submission rather than kept as demonstration content, because the University data they need was not available; lost and found became a shared board on the backend instead (Section 4.3.5). Guest mode (FR15) was added late in the project, once the main features were working. Apple sign-in and password reset were left out of scope, and the sign-in screens show no options for them.

3.5 Experimental Setup

3.5.1 Development Tools and Environment

Table 3.5: Development tools and environment

Category	Tool / technology	Purpose
Mobile client	Flutter (Dart), Riverpod, Dio	Cross-platform app for Android and web; state management; HTTP client
Backend	Node.js, Express, Zod, bcryptjs, jsonwebtoken	REST API; request validation; password hashing; session tokens
Database	PostgreSQL	Relational storage (Table 3.4)
External services	Firebase Authentication and Cloud Messaging (firebase-admin); Google Gemini API; OpenFreeMap street map (OpenStreetMap data) through MapLibre; FOSSGIS foot-routing server (OSRM)	Google Sign-In; push notifications; AI assistant and past-paper extraction; campus map; walking times between classes
Testing	flutter_test, node:test, end-to-end smoke script, GitHub Actions	Automated tests at three levels (Section 3.5.3); the backend unit tests and the Flutter analysis and tests also run automatically on every push
Tools	Visual Studio Code, Android Studio and Android emulator, Git, Google Chrome	Editing; Android builds and emulation; version control; web testing
Deployment	Render (render.yaml) and Neon	Hosting for the API (Render) and the PostgreSQL database (Neon)
Hardware	Windows 11 laptop, 7.7 GB RAM	Development and testing

3.5.2 Development Methodology

The system was built iteratively and incrementally, following the Agile principle of delivering working software in small steps (Beck et al., 2001). For each feature, the backend routes and database schema were written and tested first, followed by the mobile screens, state management and widget tests, so that the server's behaviour was settled before the client was written against it. In the mobile application, a repository layer handles API and storage access (for example, `AuthRepository`, which wraps an `AuthApi` and a token store), while screen state is held in Riverpod `AsyncNotifier` and `StateProvider` controllers. This keeps networking code out of the screens and lets tests replace a repository with a fake. Rules that

the backend and the mobile application must agree on are defined once: the University of Ghana grading scale lives in `docs/grading-scale-ug.json`, and both the backend tests and the Flutter parity test read that file, so the two cannot drift apart on how grades are computed (Section 1.2). Data that the student owns and edits often, such as the timetable and exams, is stored on the device. The server's timetable routes still work but are not used by the application, because routing the dashboard through the usually empty server timetable was found during development to risk hiding a student's real classes.

3.5.3 Testing Strategy

Testing was carried out at three levels (Sommerville, 2016). Offline unit tests (`node:test` on the backend and `flutter test` on the mobile application) run without a network or database and check logic such as the grading-scale calculations, the push fan-out logic (using in-memory fakes) and the behaviour of individual screens. Database integration tests run against a real local PostgreSQL instance to catch problems that a fake database would hide, such as a uniqueness-constraint violation. End-to-end smoke tests run last, against the running API and database, and follow realistic user journeys such as registering, logging in, creating and registering for an event, and taking a quiz, in the same way a real client would.

The tests themselves were checked by mutation testing. For important business rules (event-deletion ownership, self-registration never granting an elevated role, the session-expiry flag, the Google sign-in cancel path, and the checks that keep a guest from RSVPing or joining a club), the real logic was deliberately broken, the relevant tests were confirmed to fail, and the code was then restored and checked again. A test that does not fail when the code is broken gives no real assurance, so this step was treated as essential.

3.5.4 Evaluation Criteria

The system was evaluated against five criteria: functional correctness (the automated tests in Section 3.5.3 pass); objective coverage (each specific objective in Section 1.4.2 is traced to evidence, Table 4.2); security effectiveness (role enforcement and self-registration rules are tested, and Google sign-in tokens are verified against the live identity provider); resilience (offline and partial-failure behaviour is tested against simulated failures); and end-to-end behaviour (the application is run as a whole, not only screen by screen).

On 24 October 2026 the backend unit suite passed 50/50, the database suite 14/14 and the end-to-end smoke suite 182/182; the Flutter suite passed 366/366, and `flutter analyze` reported

no issues. Two checks were done by hand. First, an invalid Google sign-in token was sent to the live Firebase project to confirm that it is rejected as invalid (HTTP 401) and not as unconfigured (HTTP 503), which shows that the credential itself is live and correctly connected. Second, a debug Android build was produced to confirm that the push-notification and Google Sign-In packages compile and link on the device. These figures are a snapshot; Chapter Four reports a fresh run.

3.5.5 Statistical Analysis

The evaluation uses descriptive counts only, namely how many tests and checks pass or fail, because the question is whether the system behaves as specified, not whether one group or design performs better than another. No hypothesis is tested, so no inferential statistics such as significance tests, confidence intervals or effect sizes are reported. This study does not include a user survey; Chapter Five recommends one.

CHAPTER FOUR
EXPERIMENTAL RESULT AND DISCUSSION

4.1 Introduction

This chapter presents and discusses the implementation and testing results of Campus Companion, a mobile application designed to integrate academic management and campus-life services for university students in Ghana. The chapter examines the major features implemented, the outcomes of automated testing, the extent to which the project objectives have been achieved, and the limitations identified during development.

The evaluation is based on results recorded on 5 and 6 November 2026 from the Flutter mobile application tests, backend unit tests, database integration tests, API smoke tests and static analysis, together with defects found by running the complete application. These results provide evidence of the behaviour of the software under the tested conditions. They do not, however, establish that every feature has been validated in a production environment or that the application has improved students' academic performance.

4.2 Overview of the Implemented System

Campus Companion was developed as a Flutter mobile application supported by a Node.js and Express backend and a PostgreSQL database. It brings several academic and campus-related functions into one application.

The implemented features include authentication with role-based access, Google Sign-In, a guest mode, an academic dashboard, timetable management, examination information, grade calculations, academic analytics with a semester-by-semester breakdown and a what-if grade projection, quizzes, past-paper question extraction, an AI Assistant, an AI Tutor that can read uploaded lecture slides, a Lost & Found board, campus locations, events, clubs, announcements, and a digital identification card.

The implementation also includes local-first functionality for selected academic features, calendar export, local reminders, an offline fallback for supported AI interactions, and a light or dark appearance that the student can choose or leave to follow the phone. Firebase Cloud Messaging and Google Sign-In have been implemented and were confirmed on a physical phone (Section 4.6.4).

The results indicate that Campus Companion has progressed beyond an interface-only demonstration. It contains functional mobile and backend components and has undergone several forms of automated testing. Nevertheless, the maturity of individual modules varies, particularly where backend integration or real-device verification remains outstanding.

The screenshots in this chapter (Figures 4.1 to 4.3) show the application running against the local backend and database, signed in as the seeded demonstration student. The development machine (7.7 GB of memory) could not keep an Android emulator responsive for a full walkthrough, so the screenshots were taken from the same codebase compiled as a web build and shown in a phone-sized (412 × 915) browser window. Flutter draws every screen from the same widget code on both targets, so the screens are the same apart from the Android status bar. The only values entered by hand are the fictional student ID, level and programme in the optional profile, which feed the digital identification card. The Android build itself was checked on a Samsung Galaxy A16 running Android 16 (Section 4.6.4).

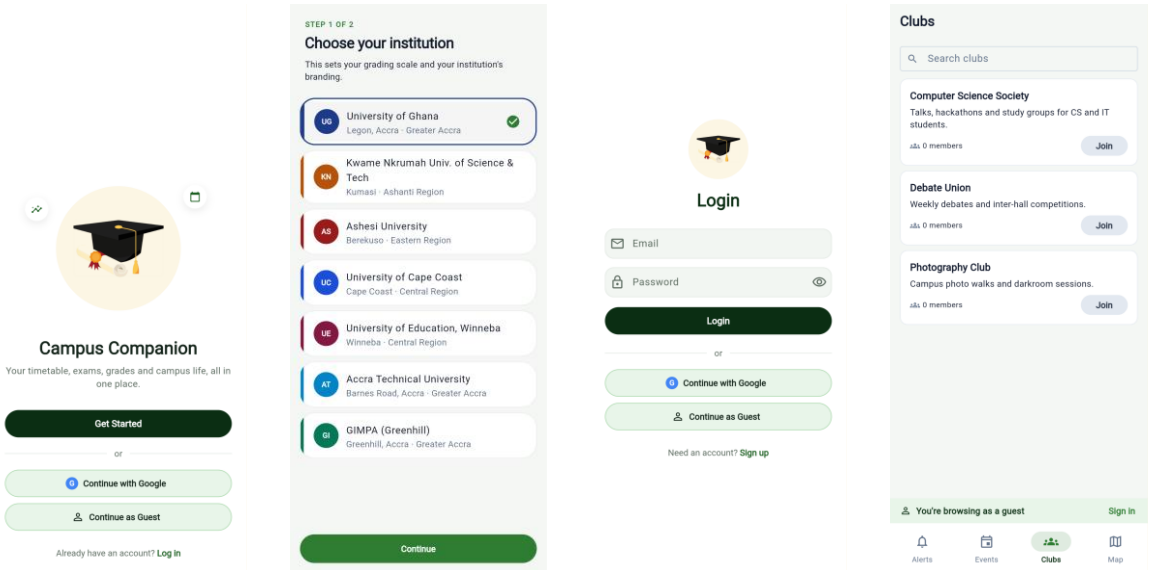
4.3 Results of Feature Implementation

4.3.1 Authentication and Onboarding

Users register and sign in with an email address and password; passwords are stored only as bcrypt hashes, and sessions use JSON Web Tokens. Three roles are supported: students register themselves, while lecturer and administrator accounts can only be created by an administrator. Role checks are enforced by the backend on every protected request, not just hidden in the interface.

On first launch the student sees a Welcome screen, chooses an institution, which sets the grading scale and branding, and may add optional profile details for the digital identification card (Figure 4.1). Google Sign-In is implemented end to end, its backend verification has been checked against the live Firebase project, and a real sign-in on a physical phone created the student's account (Section 4.6.4). A visitor can also choose “Continue as Guest”. Guests see four tabs (alerts, events, clubs and the campus map), served by read-only backend routes that need no token, and are told they are browsing as a guest (Figure 4.1(d)). The choice is remembered on the device, so a guest who reopens the application returns to the guest view until they sign in or choose to leave it. Every personal feature is left out, and tapping RSVP or Join asks the guest to sign in instead of sending a request. A request that carries an invalid token is still rejected, so a signed-in student whose session expires is not silently treated as a guest. The sign-in screens offer only what works: there is no Apple sign-in or password-reset

option, and if Google sign-in is tapped in a build where it has not been switched on, the application says so and suggests email and password instead.



(a) Welcome (b) Institution picker (c) Login (d) Guest browsing

Figure 4.1: Onboarding, sign-in and guest mode

4.3.2 Academic Management

The academic management features were implemented to help students organise class schedules, prepare for examinations, and understand their academic results.

The timetable module displays schedules and supports local-first handling. Calendar export allows supported schedule information to be exported in the iCalendar format, while local notifications support reminders. Examination-related features provide access to examination information and preparation activities.

The grade management module calculates grades and grade point averages using the University of Ghana grading scale, which matches the University's regulations: A (80 to 100) carries 4.0 grade points and D (50 to 54) 1.0, while E (45 to 49) is a failure grade that still carries 0.5 (University of Ghana, 2017, Section 9.20). A grading inconsistency discovered during development was corrected so that the relevant mobile and backend calculations interpret supported score ranges consistently. Figure 4.2(b) shows the result for the demonstration student: a cumulative GPA of 3.64, the same figure the backend's grading tests expect for this student, and First Class Honours from an FGPA that is also 3.64, since all of the student's courses are at Level 200.

This correction demonstrates the importance of validating business rules across different components of the same application. If the interface displays a grade that differs from the grade used in GPA calculations, users may receive misleading academic information. Consistent grading logic reduces this particular risk.

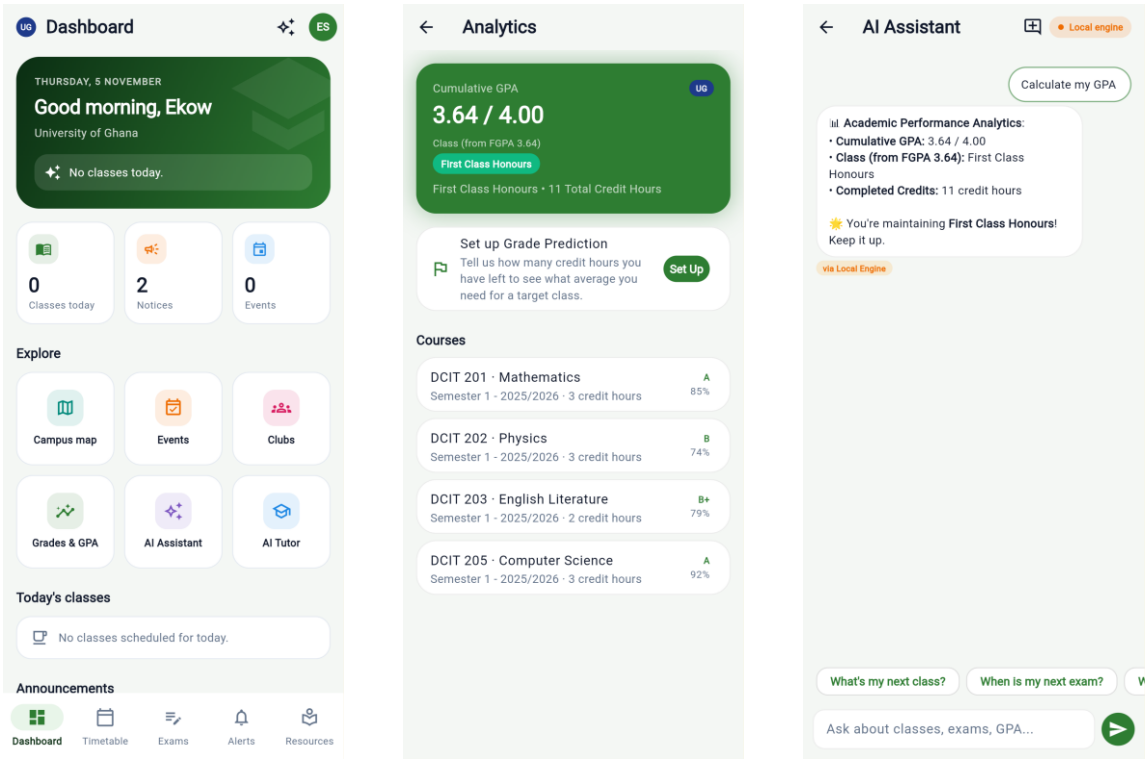
The application also provides academic summaries and a target-class calculator, which works out the average a student needs in their remaining credit hours to reach a chosen honours class. It is a fixed formula, not a prediction model. Its honours-class bands match the regulations (First Class 3.60 to 4.00 down to Pass 1.00 to 1.49). The University classifies a degree on the final grade point average (FGPA), in which the GPAs of Levels 100 to 400 are weighted 1:1:2:2 (University of Ghana, 2017, Section 9.31), so the application takes the class from the FGPA rather than the cumulative GPA. It reads each course's level from the first digit of its course number (Section 9.13), averages each level and combines the levels with those weights. A student whose results differ between levels can therefore receive a different class from the one the cumulative GPA would suggest, and the tests include transcripts where this happens in each direction. If a course's level cannot be read, the application falls back to the cumulative GPA and labels the class as an estimate. The calculator also works on the FGPA; because it cannot know which levels the remaining credits will fall in, it assumes they are spread evenly over the levels left, and the card says so. Repeated courses, which the regulations weight separately (Section 9.31c), are not treated differently.

Analytics also lists the results semester by semester, as a University transcript does: each semester shows its own GPA, the cumulative GPA up to and including it, and the credits taken. Checked against a real University of Ghana transcript, the cumulative GPAs it gives for the first semesters match the transcript's. A “What if?” screen lets a student enter the grades they expect in coming courses, starting from the timetable courses that have no result yet, and shows the cumulative GPA, FGPA and class those grades would give, with each course's level read from its number. Nothing entered there is saved; it is a planning aid, not a record.

Because the application has no connection to the University's records, students can also record their own results with “Add a result” (course code and title, semester, credits and grade). These results are saved to the student's account and count towards the cumulative GPA, FGPA and semester view, but they are labelled “Entered by you”, are shown without a mark because only the letter grade is known, and can be removed by the student. The server stores them separately from official grades: it refuses to let a self-entered result overwrite an official one, and a student

can never change another student's results. Both rules were checked on the deployed server on 6 November 2026.

A limitation of the current timetable implementation is that it relies on device-local data as its primary source. The backend schedule route is not currently used by the application as a synchronised backup. As a result, complete cross-device timetable synchronisation has not been demonstrated.



(a) Dashboard (b) GPA analytics (c) AI Assistant

Figure 4.2: Academic features

4.3.3 Quiz and Past-Paper Features

The quiz module supports the creation and completion of quizzes. It also supports the import of past-paper PDFs and Gemini-assisted extraction of questions from supported documents. The quizzes are opened from the AI Tutor ("Course quizzes"), on the course the tutor is set to, next to the practice questions the tutor can generate from a student's own slides (Section 4.3.4); lecturers and administrators create them from the same screen.

The implementation prevents answer keys from being revealed before submission, and automated tests cover the relevant quiz and past-paper extraction behaviour.

These capabilities provide a foundation for examination preparation within the application. However, the quality of extracted questions may depend on document formatting, text clarity, and the structure of the original paper. Passing automated tests does not establish that every question will be extracted correctly from every PDF. Further evaluation using a varied collection of actual past papers would be required to measure extraction accuracy and identify document formats that require additional handling.

4.3.4 AI Assistant and Tutor

The AI Assistant provides a conversational interface for academic and campus-related questions. It uses Gemini when the necessary API configuration is available and has a local keyword-based fallback for supported interactions. Figure 4.2(c) shows the fallback answering "Calculate my GPA" from the student's stored grades; the reply is labelled "via Local Engine". A short check with a live Gemini key is reported in Section 4.6.3.

The assistant can retrieve relevant information from supported application areas, including timetable information, grades, campus locations, events and alerts. Context retrieval is designed so that failure to retrieve one category does not prevent the use of the other available categories; the assistant instead tells the student which information it could not reach. The AI Tutor reuses the same conversational interface for tutoring-related interactions.

The AI Tutor can also work from a course's lecture slides. The student attaches them as a PDF of up to 12 MB (PowerPoint slides are saved as PDF first); the file is sent to Gemini with each question, and the tutor is instructed to answer from the slides, mention the slide or page a point comes from, and say when the slides do not cover a question. The slides stay in the application's memory only. Every Gemini request uses Gemini 3.5 Flash-Lite, chosen for speed: in checks on 5 November 2026 it answered in about 2 to 13 seconds, against about 15 seconds for the one reply Gemini 3.5 Flash gave, and asked about a 1.2 MB PDF, both it and Gemini 3.1 Flash-Lite found the right figure and cited the page. When the model reports that it is busy, the single retry goes to Gemini 3.1 Flash-Lite instead of waiting on the same model. Several requests to each model were refused with "high demand" errors during those checks, so the local fallback remains necessary.

The implementation therefore supports both external AI-assisted interactions and a more limited fallback mode. The fallback provides some resilience when the external service is unavailable, but it does not offer the same capabilities as a full generative AI model.

The current results establish that assistant-related logic is covered by automated testing. They do not establish the factual accuracy of every AI response, the quality of explanations across all subjects, or a measurable improvement in student learning, and the assistant has been exercised against a live Gemini API key only in the short checks in Sections 4.6.3 and 4.6.4. Those outcomes require a separate evaluation using representative questions and, if educational effectiveness is being assessed, participation from intended users.

4.3.5 Campus-Life Features

Campus Companion includes campus locations, events, clubs, announcements, and digital identification features (Figure 4.3).

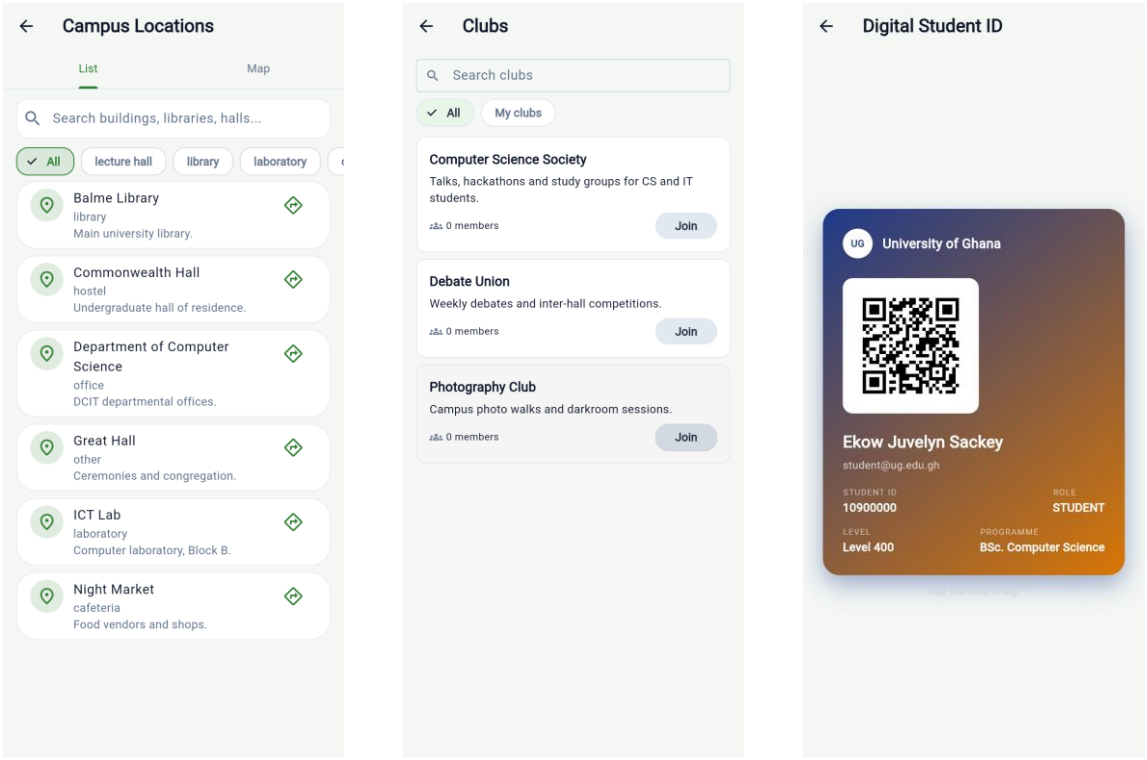
Campus locations are shown as a list organised by category and on a street map. The map uses OpenFreeMap, a free service that needs no API key, to draw OpenStreetMap streets and buildings, with each location marked in its category colour; tapping a location opens its details, from which it can be opened in an external mapping application. Because the street map needs a connection, a simple map drawn by the application itself remains available offline. The application also estimates the walking time between consecutive classes. It asks a public routing service built on the same OpenStreetMap data, the FOSSGIS foot-routing server, for the route along real footpaths, and keeps each result on the device so that a walk is fetched only once. Between Balme Library and the Great Hall the route is 339 metres, about five minutes, against 159 metres in a straight line. When no route can be fetched, the application falls back to the straight-line distance and says that the figure is an estimate.

The events module supports event creation, registration, cancellation and deletion, subject to the applicable permissions: event deletion is restricted to administrators or the lecturer who created the event. The clubs module supports club discovery, searching, viewing details, joining and leaving.

The announcements module provides an alerts feed and supports administrator broadcasts. Foreground polling is used to retrieve updates while the application is active. The digital identification module presents a flippable card with a QR code.

These modules demonstrate the breadth of the application's campus-life functionality. However, a feature's presence in the application should not be confused with complete operational integration. Lost and found is a board shared through the backend: a signed-in student posts an item they lost or found, with an optional place, details and a contact note, and

the poster (or an administrator) marks it returned or removes it. Guests can read the board, but names and contact notes are sent only to signed-in users. The server also suggests possible matches: an open “lost” post and an open “found” post whose titles share a meaningful word list each other as possible matches (a shared place ranks the match higher), and when a new post matches, the other poster receives a push notification that carries no contact details. The matching is deliberately simple and only suggests; it was checked end to end on the deployed server on 5 November 2026. The board has its own tab in the Resources section and is also reachable from the account menu. A post can carry one photo, taken with the camera or chosen from the gallery; the application shrinks it to at most 1,280 pixels before upload, and the server accepts only JPEG or PNG images of at most 2 MB, recognised by their contents rather than their names. On the deployed server a photo was uploaded and served back unchanged. The Resources section links the institution's official portals and, for the University of Ghana, its Regulations for Junior Members (University of Ghana, 2017). Earlier screens for dues with a simulated mobile-money payment, shuttle times, library seat counts, past papers, an academic calendar and document requests showed invented sample data; they were removed rather than kept as labelled demonstrations, since the University data they need was not available.



(a) Campus locations (b) Clubs (c) Digital student ID

Figure 4.3: Campus-life features

4.4 Automated Testing Results

Automated testing was used to examine application logic, backend functions, database operations and API workflows (Section 3.5.3).

4.4.1 Summary of Test Results

The results recorded on 5 and 6 November 2026 are presented in Table 4.1.

Table 4.1: Summary of automated testing results

Test suite	Tests passed	Total reported	Outcome
Flutter automated tests	449	449	Passed
Backend offline unit tests	119	119	Passed
Backend database tests	14	14	Passed
Backend API smoke tests	212	212	Passed
Total test executions	794	794	All reported executions passed

The four suites account for 794 passing test executions. This is the sum of the suite totals and does not necessarily represent 794 unique test cases. The suites have grown since the independent snapshot taken on 24 October 2026 (Section 3.5.4), which recorded 366 Flutter tests, 50 backend unit tests, 14 database tests and 182 smoke checks. The additions cover the defects and dashboard changes found during this evaluation, guest mode, the ownership rules of the server timetable routes (Sections 4.4.2 to 4.4.5), unit tests for the events, clubs, quizzes, locations and account-lookup code, the FGPA, walking routes, the Lost & Found board and its matching, the appearance setting and course-code formatting, slide-based tutoring, the semester breakdown and what-if projection, the course picker's layout on narrow screens, self-entered results and Lost & Found photos.

The results indicate that all reported test executions passed under their respective test environments. This provides evidence that the behaviours covered by those tests were functioning as expected at the time of the recorded runs. However, the results should be interpreted within the scope of the test suites. A passing test suite does not prove the absence of defects in untested scenarios, nor does it establish that every combination of user permissions, input values, network conditions and external services has been evaluated.

4.4.2 Flutter Automated Tests

The Flutter test suite recorded 449 passing tests out of 449 reported tests.

The tested areas include the grading and target-class calculations, calendar export, timetable parsing, assistant behaviour (including partial loss of data sources), campus-map and walking-route calculations, the FGPA, notification-related logic, past-paper extraction, the sign-in and onboarding screens, the dashboard shortcuts, and guest mode. Since the 24 October snapshot, tests were added for the three defects described in Section 4.4.7, the new dashboard shortcuts, guest mode and the demonstration-data labels, and one test was dropped when the Apple sign-in button was removed. The guest-mode tests drive the application from its root screen and check that a guest sees only the campus feeds, that RSVP and Join lead to a sign-in prompt without sending a request, and that a request sent without a token is never treated as an expired session. Removing either sign-in check, or the token check, was confirmed to make these tests fail. One of the new label tests also exposed a further defect: the Student Handbook card on the Resources screen raised a framework assertion in debug builds because of how it was nested, and it was fixed in the same change.

These tests help verify important parts of the mobile application's business logic. For example, testing grading calculations helps detect inconsistencies in the interpretation of score ranges, while calendar export tests help verify the creation of supported calendar data. The result provides positive evidence for the mobile functions covered by the suite. It does not independently verify the complete appearance, accessibility, usability, or behaviour of every screen on every supported physical device.

4.4.3 Backend Unit Tests

The backend offline test suite recorded 119 passing tests out of 119 reported tests.

The suite covers grading-scale behaviour, the events, clubs, quizzes, locations, Lost & Found (including its matching and photo checks), students' own results and account-lookup controllers, push-notification fan-out and sender logic, the verification of Google sign-in tokens, and the optional token check on the routes guests may read, using test doubles in place of external services. Using test doubles allows selected backend functions to be examined without requiring every test to contact a real external service. The result indicates that the tested backend logic behaved as expected under the configured test conditions. It does not establish successful communication with every external service or guarantee that the same behaviour will occur under all production conditions.

4.4.4 Database Integration Tests

The database test suite recorded 14 passing tests out of 14 reported tests against a local PostgreSQL instance.

The tests cover database operations involving device tokens and push-notification preferences, and the linking of Google sign-in identities to accounts (creating a new account, linking an existing one by email, and rejecting an attempt to link one Google account to two users). These tests are important because unit tests that use simulated dependencies cannot, by themselves, confirm that SQL operations execute correctly against a real database. The result provides evidence that the specific database operations covered by the suite executed successfully in the local test environment. It does not constitute a complete assessment of database performance, recovery, security, or behaviour under production workloads.

4.4.5 API Smoke Tests

The backend API smoke-test suite recorded 212 passing tests out of 212 reported tests against the configured local database environment.

Smoke tests provide a broad check that selected API workflows can execute without major failures. The suite walks guest browsing, authentication, role permissions, events, clubs, announcements, device registration and the quiz lifecycle. The guest checks confirm that the five public routes answer without a token and show nothing as registered or joined, that a forged token is still rejected on those routes, and that every write and every personal route still requires sign-in. Seven further checks cover the server timetable routes: an owner identifier supplied in a request is ignored, students and unauthenticated callers cannot create or delete entries, and an administrator cannot delete an entry that belongs to another user. It also includes one check against a live external service: a deliberately invalid Google sign-in token is sent to the real Firebase project and is rejected as invalid rather than as unconfigured, which confirms that the credential itself is live and correctly wired.

The recorded result supports the conclusion that the tested API workflows were functioning in the local environment. It does not demonstrate that the application has been deployed successfully to a production server, that every endpoint has been exhaustively tested, or that the backend can support a particular number of simultaneous users. The same smoke test was later run against the deployed server (Section 4.7), and all 212 checks passed.

4.4.6 Static Analysis

Flutter static analysis reported no issues in the recorded run. Static analysis helps identify certain classes of programming errors and code-quality problems without requiring every application path to be executed. The result is a useful indicator of the codebase's current static-analysis status, but it is not a substitute for runtime testing, security assessment, or user evaluation.

4.4.7 Defects Found by Running the Application

Running the complete application to capture the screenshots in this chapter exposed three defects that the passing automated suites had not caught. First, on a phone-width screen the dashboard's title was truncated to “Da…” because six unlabelled shortcut icons shared the top bar with it; the shortcuts were moved into a labelled grid on the dashboard (Figure 4.2(a)). Second, a rejected login, such as a mistyped password, replaced the whole screen with a full-page error and a Retry button, so the login form and its explanatory message disappeared: the application's root screen treated any session error as a failure to restore a saved session. Third, finishing or skipping the profile step returned the student to the institution picker instead of continuing to the login screen, because only the topmost onboarding screen was closed. All three were fixed, and the second and third are now covered by regression tests that were first confirmed to fail against the defective code.

The second and third defects are worth explaining. Each screen involved already had passing tests, because each was tested in isolation; both bugs only appeared when the screens ran inside the application's real root screen and navigation. For that reason, the new regression tests drive the application from its root rather than from the individual screen.

4.5 Evaluation Against Project Objectives

The specific objectives set out in Section 1.4.2 provide a basis for assessing the extent of implementation achieved (Table 4.2).

Table 4.2: Assessment of the specific objectives

Objective (Section 1.4.2)	Current evidence	Assessment
1. Cross-platform client and REST backend with role-based authentication, including Google Sign-In	Flutter application for Android and the web; Express API with JWT sessions, bcrypt password hashing and backend role checks covered by smoke tests; Google sign-in verification checked against the live Firebase project.	Implemented; Google Sign-In confirmed on a physical phone.

Objective (Section 1.4.2)	Current evidence	Assessment
2. Local-first timetable, examinations and alerts with .ics/CSV import and export and reminders	Timetable parsing and calendar-export tests; local reminders; alerts feed.	Implemented; cross-device timetable synchronisation is not provided.
3. Grade and GPA analytics on the University of Ghana scale, consistent between client and backend	One grading-scale definition checked by both the backend and the mobile tests; the score-55 inconsistency corrected.	Implemented; the scale and class bands match the University's regulations, and the class is taken from the FGPA (Levels 100 to 400 weighted 1:1:2:2).
4. Campus-life features: events, clubs, locations, quizzes and digital ID	Smoke tests for event permissions and the quiz lifecycle; mobile tests for clubs and the campus map; Figures 4.2 and 4.3.	Major modules implemented; lost and found is a shared backend board that suggests possible matches and carries photos, and screens that could only show sample data were removed.
5. AI assistant with a cloud model, a local fallback, and resilience to unreachable data sources	Assistant tests, including the loss of individual data sources; Figure 4.2(c).	Implemented; a live Gemini key answered in short checks (Section 4.6.3), including from an uploaded PDF (Section 4.3.4); response quality and educational effectiveness unevaluated.
6. Push notifications through Firebase Cloud Messaging	Fan-out and sender tests with test doubles; device-token database and smoke tests.	Implemented; delivery to a physical phone confirmed with the application in the background.
7. Layered automated testing, including mutation testing	794 passing test executions across four suites; clean static analysis; mutation checks; regression tests for the defects in Section 4.4.7.	Achieved for the recorded suites; no load, security or usability evaluation.

The assessment indicates that the principal software development objectives have been substantially addressed in the working prototype. However, implementation status is not the same as complete deployment readiness. Some objectives have been achieved at the level of functional implementation and automated testing, while others require additional integration, validation or real-world evaluation. This distinction is particularly important for the target-class calculator, AI-assisted learning, push notifications, Google Sign-In, and services that are not yet connected to the backend.

4.6 Discussion of Findings

4.6.1 Integration of Academic and Campus Services

One of the main outcomes of the project is the integration of several student-oriented functions into a single mobile application. Timetable management, examination information, grade analysis, quizzes, campus locations, events, clubs, announcements and digital identification are available within the same application environment. This design reduces the need to navigate separate interfaces when accessing the functions supported by the prototype.

Nevertheless, the application should not be described as a complete replacement for official university information systems. Some functions rely on device-local data, and services such as dues payment and shuttle times are not offered because their data is held by the University. Full institutional integration would require reliable data sources, appropriate synchronisation, and confirmation that the application reflects authoritative university records.

4.6.2 Consistency of Academic Calculations

The correction of the grading inconsistency demonstrates the importance of using consistent business rules throughout a software system. When grades are displayed in the mobile application and also used by backend calculations, the same grading scale should produce consistent results. Before the correction, the server graded on a simplified scale, so a score of 55 was shown as a fail while the GPA calculation counted it as a D+. The grading scale is now defined once and checked by both the backend and the mobile test suites (Section 3.5.2).

The corresponding automated tests provide evidence that the tested grading behaviour is consistent under the defined cases. Additional validation should still compare a broader range of scores, grade boundaries, semester combinations and GPA scenarios against the official grading requirements applicable to the intended institution.

4.6.3 AI Assistance and Fallback Behaviour

The AI Assistant combines application context with an optional external generative AI service and a local keyword-based fallback. This design provides two modes of interaction and reduces dependence on the external service for every supported assistant operation. However, the fallback is limited and should not be treated as a full substitute for generative AI. Similarly, the availability of contextual information does not guarantee that every generated answer is correct or appropriate.

A Gemini API key was configured on 4 November 2026, and the live check (`npm run verify:integrations`) confirmed that the Gemini API accepts it. On the web build, signed in as a test student, the assistant was asked how to plan revision for examinations two weeks away. The answer came from Gemini, not the local engine: it carried no “via Local Engine” label and referred to the Balme Library, the Great Hall and the application's Exams tab. A follow-up question then received a temporary “high demand” error (HTTP 503) from the Gemini service, and the application answered from the local engine instead, labelling the reply accordingly, which is the intended fallback. Gemini's replies took 20 to 27 seconds, close to the application's original 30-second limit, so the application now waits up to 60 seconds and retries once when Gemini reports that it is busy or rate-limited (HTTP 503 or 429). Tests cover the retry, and the Flutter suite passed 384 of 384 tests after the change. This was a single short check; it does not evaluate the quality of the answers.

Future evaluation should measure response correctness, relevance, response time, failure handling, and the extent to which users find the assistant helpful. Any claims about improved learning outcomes would require a suitable user study rather than relying on implementation or unit-test results alone.

4.6.4 Reliability of Notification and Sign-in Integrations

The project includes notification-related functionality at both the application and backend levels. Automated tests cover selected notification logic, and database tests cover device-token and preference operations. The Firebase project is configured, and a live check run on 3 November 2026 (`npm run verify:integrations`) confirmed three things: the backend's credential loads, Firebase Cloud Messaging accepts a dry-run message sent with it (the message is validated and authorised but not delivered), and real Firebase rejects an invalid sign-in token. A second run on 4 November 2026 verified four checks with none failing: Google is now enabled as a sign-in provider, with the application's signing fingerprint registered, and the Gemini API accepts the configured key. On the Android emulator a Google sign-in timed out before an account could be chosen, so both integrations were then checked on a Samsung Galaxy A16 running Android 16, connected by USB to the development machine running the local backend.

On that phone, signing in with Google created a new student account on the server, with no password, and three seconds later the application registered the phone's notification token. With the application in the background, an administrator then posted a test announcement; it arrived

as a notification on the application's announcements channel, and the student saw it on screen. (A first attempt was lost because the development server restarted just after the announcement was saved, before the notification had been sent; the second was delivered.) The same build also showed the OpenFreeMap street map with all six locations, and the assistant answered from Gemini rather than the local engine. These checks used one phone, one Google account, the debug signing key and the local backend; a notification arriving while the application is open was not observed, and nothing has yet been tested against a deployed server. The test announcements were deleted afterwards.

4.6.5 Local-First Operation and Synchronisation

Local-first handling supports selected functions without requiring every operation to depend on a backend request. This can be useful in environments where connectivity is intermittent. However, local availability and data synchronisation are separate properties. The current timetable implementation treats device-local data as the primary source and does not use the backend schedule route as a synchronised backup. Therefore, the project has not established that timetable changes automatically propagate between devices or remain consistent across multiple sessions and installations.

Future work should define a clear source of truth, synchronisation rules, conflict handling and recovery behaviour before complete multi-device timetable synchronisation can be claimed.

4.6.6 Technical Quality and Test Coverage

The recorded automated test results are encouraging because they cover mobile logic, backend functions, database operations and API workflows, and clean Flutter static analysis further supports the current state of the codebase. Every backend module now has dedicated unit tests. Events, clubs, quizzes, locations and the account lookup used by registration and login were at first exercised only by the end-to-end smoke suite, which gives a less precise failure location; in the final review they gained 40 unit tests that run the controllers against a scripted stand-in for the database, so they need neither a database nor a network. They check, for example, that a lecturer cannot delete another lecturer's event, that a quiz's answer key is not sent before an attempt, that a new account is always a student, and that a wrong password and an unknown email receive the same reply. Deliberately breaking the ownership check or the answer-key rule makes the matching test fail.

The tests also do not cover every aspect of system quality. The project has not established formal load-test results, measured response-time benchmarks, a representative usability study,

or a production security assessment. The defects in Section 4.4.7 show a further gap: screens that each passed their own tests still failed when combined. The results should therefore be interpreted as evidence of functional correctness for the tested scenarios rather than proof of comprehensive reliability, scalability, security or user satisfaction.

4.7 Outstanding Issues and Deployment Readiness

Although the application contains a substantial set of implemented features, several items remain important before it can be considered ready for production use (Table 4.3).

Table 4.3: Outstanding issues and required verification

Issue	Current state	Required action
Push notifications	Delivered to a physical phone with the application in the background, from the local backend.	Check a notification arriving while the application is open, and repeat on the deployed server.
Google Sign-In	Completed on a physical phone (4 November 2026), with the debug signing key, which is the only fingerprint registered.	Register the release signing key's fingerprint before the application is published.
Deployment	The API was deployed on 5 November 2026 to Render's free plan, with a free Neon PostgreSQL database; the full smoke test passed 212/212 against it, with Google sign-in enabled. Free plans sleep when idle, and the seeded accounts use random passwords rather than the local ones.	Move to paid plans for production use, add backups and monitoring, and repeat the Google sign-in and push checks against the deployed server.
Repository setup	The repository is backed up to a private GitHub remote (3 November 2026). A GitHub Actions workflow runs the backend unit tests and the Flutter analysis and tests on every push; its first run passed (4 November 2026). The database and smoke tests run locally only, and there is no deployment workflow.	Run the database and smoke tests in the workflow against a test database, and add a deployment workflow.
Timetable synchronisation	The mobile application uses local-first timetable data.	Define and implement backend synchronisation if cross-device consistency is required.
University services	Dues payment, shuttle times, library occupancy and the academic calendar are not offered; the earlier sample versions were removed.	Obtain the data, or access to the University's systems, and a payment provider before adding them.
Degree class	Taken from the FGPA (Levels 100 to 400 weighted 1:1:2:2; University of Ghana, 2017). Repeated courses are not weighted separately (Section 9.31c), and the target-class calculator assumes the remaining credits are spread evenly over the levels left.	Handle repeated courses, and let students state how many credits remain at each level.

Issue	Current state	Required action
AI response quality	A live Gemini key answered in a short check on 4 November 2026 (Section 4.6.3), but response quality has not been evaluated.	Test representative questions, factual correctness, failure handling and usefulness.
Security and operational readiness	Core authentication and permission mechanisms are implemented.	Conduct a production security review and verify logging, backups, configuration and data protection.
User evaluation	No formal user-study results are available.	Conduct usability testing with appropriate participants and document the findings.

These issues do not negate the implementation work completed. Instead, they identify the difference between a technically substantial working prototype and a fully validated production application. The most immediate priorities are to complete end-to-end notification and sign-in verification, move the deployment beyond free hosting plans, confirm backup and configuration procedures, and address the most important integration gaps. The quality of the AI assistant and the application's usability should then be checked using defined methods and measurable criteria.

4.8 Limitations of the Evaluation

The evaluation presented in this chapter is primarily based on software implementation and automated testing. It does not include measured academic outcomes, a controlled comparison with existing student applications, or results from a representative survey of university students.

No measured response-time or load-capacity figures are reported because such results have not been established. Similarly, the application has not been shown to improve GPA, reduce academic stress, increase productivity or improve examination performance.

The findings are also limited by device checks on a single phone against the local backend, the local-first timetable design, partial backend integration of selected services, and screenshots taken from the web build rather than a physical device. The backend is deployed on free hosting plans, but its behaviour under real use remains to be verified. These limitations should be considered when interpreting the project's achievements; they also provide a clear direction for future work.

4.9 Chapter Summary

This chapter presented the results of implementing and testing Campus Companion. The application provides a range of academic and campus-life functions, including timetable and examination management, grade calculations, academic analytics, quizzes, AI assistance including tutoring from uploaded slides, a Lost & Found board, campus locations, events, clubs, announcements and digital identification.

The results recorded on 5 and 6 November 2026 show 449 passing Flutter tests, 119 passing backend offline unit tests, 14 passing database tests and 212 passing API smoke tests, 794 passing test executions in total, and Flutter static analysis reported no issues. Running the complete application exposed three defects the suites had missed, which were fixed and are now covered by regression tests.

The findings indicate that the principal development objectives have been substantially addressed at the working-prototype level. However, the results do not establish complete production readiness or demonstrate improved educational outcomes. Production-grade hosting, timetable synchronisation, selected backend integrations and formal evaluation of the AI assistant remain outstanding. The next chapter presents the overall conclusions of the project, summarises its contributions, and recommends improvements for future development.

CHAPTER FIVE
CONCLUDING REMARKS

5.1 Summary of the Study

This project set out to address a practical problem: Ghanaian university students use several unconnected tools to manage their academic and campus lives, and none of these tools follows their institution's own rules, such as the University of Ghana grading scale (Section 1.2). In response, it designed, built and tested Campus Companion, a Flutter mobile application with a Node.js, Express and PostgreSQL backend. The application covers authentication with three user roles, a timetable and examinations module that works offline, GPA analytics on the University of Ghana scale, events, clubs, campus locations, announcements, quizzes, a digital student ID and an AI assistant.

The requirements came from the developer's own analysis of students' needs (Section 3.1). The system was built one feature at a time, with the backend for each feature written and tested before the mobile screens, and was checked with offline unit tests, database tests, end-to-end API smoke tests and mutation testing (Chapter Three). In the final test runs, all 449 Flutter tests, 119 backend unit tests, 14 database tests and 212 API smoke checks passed (Chapter Four). Measured against the seven specific objectives, most were met at the working-prototype level, and Google Sign-In and push notifications were confirmed on a physical phone; the AI assistant's answers from the live model have been checked only briefly and not evaluated (Table 4.2).

5.2 Contribution of the Study

The contributions claimed in Section 1.5 are supported as follows. First, none of the systems reviewed in Chapter Two combines a student's academic data with campus-wide services under one account while applying local grading rules (Table 2.1); Campus Companion does, using the actual University of Ghana scale. Second, the AI assistant reports which of its data sources it could not reach instead of failing the whole answer. The chatbot studies reviewed in Section 2.6 focus on whether chatbots help learning, not on this kind of robustness, so the design adds a practical point to that discussion. Third, the testing approach is documented with repeatable results (Chapter Four), including the defects that only appeared when the complete application was run (Section 4.4.7).

5.3 Limitations

Several limitations should be stated together. Google Sign-In and push notifications were confirmed on one phone, with the debug signing key and the local backend; a notification arriving while the application is open was not observed, and the release signing key is not yet registered (Section 4.7). The backend is deployed on free hosting plans and passed the smoke test there, and the application has been used against it on a second phone (a Xiaomi Redmi Note 13); Google sign-in and push notifications have not yet been repeated against the deployed server, and no security review has been carried out. The AI assistant's handling of unavailable data has been tested with simulated failures, and a live Gemini key has been used only in short checks (Sections 4.6.3 and 4.6.4). Apple sign-in and in-app password reset were left out of scope, and the application offers neither. Services whose data the University holds (dues payment, shuttle times, library occupancy and the academic calendar) are not offered (Section 1.3).

The target-class calculator is a formula based on the student's current results and remaining credit hours, not a validated prediction model, and on the FGPA it assumes the remaining credits are spread evenly over the levels left; repeated courses are not weighted separately (Section 4.3.2). Walking times depend on a public routing service; when it cannot be reached and no route has been saved, the application falls back to a straight-line estimate. The timetable is stored on the device and is not synchronised with the backend, the application has not been deployed to a production server or given a security review, and there is no iOS version.

Testing also has gaps: there are no load-capacity or security tests (Section 4.6.6). A defect in the server timetable routes was found and fixed during the final review: an administrator could delete any user's timetable entry by its identifier. Entries are now created and deleted only for the authenticated user, and smoke tests check this (Section 4.4.5). The mobile application still does not use these routes.

A formal usability evaluation involving representative university students was not conducted during this phase of the project. Although the application underwent functional and technical testing, these tests do not independently establish user satisfaction, ease of use or overall usability. Future work should include usability testing with students to gather feedback, identify usability challenges and guide further improvements to the application.

Because the study did not involve human participants, no ethics approval was sought. The usability study recommended below would need appropriate ethical approval before it is carried out.

5.4 Recommendations for Future Work

1. Register the release signing key, then repeat the Google Sign-In and push-notification checks against the deployed server, including a notification that arrives while the application is open (Section 4.7).
2. Evaluate the AI assistant's Gemini answers for correctness and usefulness against a set of representative questions.
3. Add dues payment, shuttle times and library occupancy once the University's data, or access to its systems, and a payment provider are available.
4. Add a password-reset flow and, if an iOS version is built, Apple sign-in.
5. Conduct usability testing with university students, after obtaining ethical approval, to gather feedback, identify usability challenges and guide further improvements to the application.
6. Test complete user journeys through the application's root screen, not only individual screens, since two of the three defects in Section 4.4.7 appeared only in the running application.
7. Add timetable synchronisation through the existing server routes, which now accept only each user's own entries, if students need their timetable on more than one device.
8. Move the deployed backend to production-grade hosting after a security review, and consider an iOS version.

5.5 Conclusion

Campus Companion shows that the scattered academic and campus-life tools described in Section 1.2 can be brought together in one application that follows the University of Ghana's rules. Its design keeps the student's own timetable on the device, trusts identity and roles only after the server has verified them, and keeps the AI assistant useful when some data cannot be reached. The system's behaviour is supported by repeatable automated tests, and its gaps are documented openly (Section 5.3). Most of the remaining work concerns deployment, device verification and evaluation with real users rather than redesigning the system.

REFERENCES

Adzifome, N. S., & Agyei, D. D. (2022). Learning with mobile devices: Insights from a university setting in Ghana. Education and Information Technologies. https://doi.org/10.1007/s10639-022-11300-4

Afful, D., & Boateng, J. K. (2023). Mobile learning behaviour of university students in Ghana. Cogent Social Sciences, 9(1), Article 2204712. https://doi.org/10.1080/23311886.2023.2204712

Baras, K., Soares, L., Vale Lucas, C., Oliveira, F., Paulo, N. P., & Barros, R. (2018). Supporting students' mental health and academic success through mobile app and IoT. International Journal of E-Health and Medical Communications, 9(1), 50–64. https://doi.org/10.4018/IJEHMC.2018010104

Beck, K., Beedle, M., van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., Grenning, J., Highsmith, J., Hunt, A., Jeffries, R., Kern, J., Marick, B., Martin, R. C., Mellor, S., Schwaber, K., Sutherland, J., & Thomas, D. (2001). Manifesto for Agile software development. https://agilemanifesto.org/

Chen, E., Pei, Y., Chen, S., Tian, Y., Kotcher, R., & Tague, P. (2014). OAuth demystified for mobile application developers. In Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security (CCS '14) (pp. 892–903). ACM. https://doi.org/10.1145/2660267.2660323

Essel, H. B., Vlachopoulos, D., Tachie-Menson, A., Johnson, E. E., & Baah, P. K. (2022). The impact of a virtual teaching assistant (chatbot) on students' learning in Ghanaian higher education. International Journal of Educational Technology in Higher Education, 19, Article 57. https://doi.org/10.1186/s41239-022-00362-6

Fielding, R. T. (2000). Architectural styles and the design of network-based software architectures [Doctoral dissertation, University of California, Irvine]. https://ics.uci.edu/~fielding/pubs/dissertation/top.htm

Hardt, D. (Ed.). (2012). The OAuth 2.0 authorization framework (RFC 6749). Internet Engineering Task Force. https://doi.org/10.17487/RFC6749

Jones, M., Bradley, J., & Sakimura, N. (2015). JSON Web Token (JWT) (RFC 7519). Internet Engineering Task Force. https://doi.org/10.17487/RFC7519

MyStudyLife. (n.d.). MyStudyLife: The ultimate student productivity tool. Retrieved October 31, 2026, from https://mystudylife.com/

Okonkwo, C. W., & Ade-Ibijola, A. (2021). Chatbots applications in education: A systematic review. Computers and Education: Artificial Intelligence, 2, Article 100033. https://doi.org/10.1016/j.caeai.2021.100033

OWASP Foundation. (2021). OWASP Top 10: 2021. https://owasp.org/Top10/

Provos, N., & Mazières, D. (1999). A future-adaptable password scheme. In Proceedings of the 1999 USENIX Annual Technical Conference, FREENIX Track (pp. 81–91). USENIX Association.

Samarin, N., Sanchez, A., Chung, T., Juleemun, A. D. B., Gilsenan, C., Merrill, N., Reardon, J., & Egelman, S. (2024). The medium is the message: How secure messaging apps leak sensitive data to push notification services. Proceedings on Privacy Enhancing Technologies, 2024(4). https://doi.org/10.56553/popets-2024-0151

Sommerville, I. (2016). Software engineering (10th ed.). Pearson.

University of Ghana. (n.d.). Sakai learning management system. Retrieved October 31, 2026, from https://sakai.ug.edu.gh/

University of Ghana. (2017). Regulations for junior members (Vol. 1). https://www.ug.edu.gh/sites/default/files/UG_Regulations_for_Junior_Members-2017.pdf

Wang, Y., & Yan, X. (2018). Research and implementation of wisdom campus system based on Android platform. In Proceedings of the 2017 7th International Conference on Education and Management (ICEM 2017) (pp. 151–154). Atlantis Press. https://doi.org/10.2991/icem-17.2018.33

Wollny, S., Schneider, J., Di Mitri, D., Weidlich, J., Rittberger, M., & Drachsler, H. (2021). Are we there yet? A systematic literature review on chatbots in education. Frontiers in Artificial Intelligence, 4, Article 654924. https://doi.org/10.3389/frai.2021.654924

Zhao, Y., & Pardo, A. (2017). Evolving the design of a mobile application to support transition to tertiary education. Frontiers in ICT, 4, Article 25. https://doi.org/10.3389/fict.2017.00025